

Neo4j TestPilot: Technical Q&A

This document provides a list of deep technical questions and answers about the Neo4j TestPilot project, designed to help you prepare for a technical interview.

Model Architecture

Q1: Why did you choose Neo4j, a graph database, over a traditional relational (SQL) database for this project?

A: I chose Neo4j for three primary reasons. First, **natural relationship modeling**; a graph database is inherently suited to representing complex relationships like class inheritance, dependencies, and documentation links, which are central to this project. Second, the **power of Cypher queries**; expressing complex queries, such as finding all subclasses of a given class up to a certain depth, is significantly more concise and intuitive in Cypher than in SQL. Finally, the **graph visualization capabilities** of the Neo4j Browser were invaluable for exploring and understanding the structure of the legacy codebase.

Q2: Can you walk me through the 8-phase ingestion pipeline?

A: The 8-phase pipeline was designed to be a modular and maintainable way to process the Java codebase. It starts with **Project Structure** to identify modules and dependencies. Then, **Class Parsing** uses Tree-sitter to parse the Java files. **Members Extraction** pulls out methods and fields. **Dependency Graph** and **Import Resolution** build the core dependency map. **Git Versioning** adds a temporal dimension to the data. **Semantic Changes** uses an LLM to find breaking changes between versions. Finally, **Markdown Linking** connects documentation to the code.

Q3: Describe the architecture of the MCP integration with GitHub Copilot.

A: The MCP integration allows GitHub Copilot to communicate with the Neo4j database. It consists of a Go-based **GenAI Toolbox** that acts as an MCP server. This server exposes custom tools defined in a YAML file. When a user makes a request in VS Code, the MCP client sends it to the server, which then translates it into a Cypher query

for Neo4j. The query results are returned to Copilot, which uses them to generate the final code.

Data Challenges

Q4: What were the main challenges you faced when parsing the legacy Java codebase?

A: The biggest challenge was the sheer complexity and, at times, inconsistency of the legacy code. An initial attempt with a regex-based parser failed on about 12% of the files due to complex syntax, anonymous classes, and other edge cases. To overcome this, I switched to the **Tree-sitter** framework, which is a much more robust and error-tolerant parser. This allowed me to achieve a 100% parse success rate across all 481 classes.

Q5: How did you handle tracking and comparing different versions of the codebase?

A: I integrated Git directly into the ingestion pipeline. Each ingested entity (class, method, etc.) was tagged with its corresponding Git commit, tag, and branch. For version comparison, I developed a **semantic change detection** module that used an LLM to analyze the output of `git diff` between two versions. This allowed the system to identify not just syntactic changes, but also semantic ones, like modified method signatures or removed imports.

Ingestion & Optimization

Q6: The initial ingestion time for a new version was 34 minutes. How did you identify the bottlenecks and what optimizations did you implement?

A: Through profiling, I identified two main bottlenecks. The first was the heavy use of `MERGE` operations in Neo4j, which are slower than `INSERT`. I optimized this by implementing **batch processing**, grouping 50 classes into a single transaction to reduce network overhead. The second bottleneck was the LLM-powered semantic analysis. I addressed this by using a `ThreadPoolExecutor` to **parallelize the LLM API**

calls, and also made the semantic analysis step optional for faster re-runs when breaking change detection wasn't needed.

Metrics & Quality

Q7: How did you measure the success and impact of the Neo4j TestPilot project?

A: I used a combination of quantitative and qualitative metrics. The most significant quantitative metric was the **18x reduction in test generation time**, from 2-3 hours to under 10 minutes. I also measured the **code quality** of the generated code, which had zero linting errors, and the **documentation completeness**, which reached 100% Javadoc coverage. Qualitatively, the project enabled a new AI-assisted development workflow and created a valuable, queryable knowledge graph of the entire codebase.

Q8: How did you ensure the quality and correctness of the AI-generated code?

A: I implemented a multi-layered quality assurance strategy. First, the code generation prompts were engineered to enforce strict coding standards, such as the use of explicit imports and comprehensive Javadoc comments. Second, the use of versioned templates from Neo4j ensured consistency with the target codebase version. Finally, all generated code was validated to ensure it compiled without errors and passed all checkstyle rules.

Production & Deployment

Q9: How is the MCP server deployed and managed in a production environment?

A: The MCP server is a lightweight, self-contained Go executable. It's deployed as a `stdio` server, meaning it's managed by the developer's local VS Code instance. This simplifies deployment as there's no need for a separate, network-accessible server. The configuration is handled through a local `mcp.json` file in the user's VS Code settings.

Challenges & Future Improvements

Q10: What was the single biggest technical challenge you faced, and how did you overcome it?

A: The biggest technical challenge was achieving a 100% parse success rate on the legacy Java codebase. As mentioned, the initial regex-based approach was too brittle. The solution was to adopt the **Tree-sitter** parsing framework. This required learning its API and query syntax, and building custom logic to traverse the Java Abstract Syntax Tree (AST). It was a significant investment, but it paid off by creating a robust and reliable parsing engine.

Q11: If you had more time, what are the top three improvements you would make to the Neo4j TestPilot system?

A: First, I would implement **incremental ingestion**, so that only changed files are re-processed, rather than the entire codebase. Second, I would create a **test generation feedback loop**, where the quality of the generated code is used to automatically improve the generation prompts. Third, I would explore **distributed processing** for the semantic analysis to further reduce the ingestion time for large version changes.